

Proyecto Final

Procesamiento de Lenguaje Natural

Clasificación de Sentimientos en Reseñas de Películas con
Modelos Clásicos y Redes Neuronales Sencillas

Autor:

Alberto López Gutiérrez

Profesora:

Layla Schelli

Postgrado en Inteligencia Artificial y Machine Learning

IEBS Digital School

Julio 2026

Tabla de contenido

INTRODUCCIÓN	3
OBJETIVO GENERAL	4
OBJETIVOS ESPECÍFICOS	4
METODOLOGÍA	5
DESCRIPCIÓN DEL DATASET	6
DESCRIPCIÓN GENERAL	6
<i>Relevancia del corpus</i>	7
<i>Limitaciones del dataset</i>	7
DESARROLLO	7
FASE 1: PREPARACIÓN DE LOS DATOS	7
<i>Objetivo</i>	7
<i>Actividades</i>	8
<i>Implementación</i>	8
<i>Resultados</i>	9
<i>Interpretación</i>	11
FASE 2: ENTRENAMIENTO DE MODELOS	12
<i>Objetivo</i>	12
<i>Actividades</i>	12
<i>Implementación</i>	13
2.1. NAIVE BAYES	13
2.2. RED NEURONAL	14
<i>Resultados</i>	15
<i>Interpretación</i>	16
FASE 3: EVALUACIÓN COMPARATIVA	16
<i>Objetivo</i>	16
<i>Actividades</i>	16
<i>Implementación</i>	17
<i>Resultados</i>	17
<i>Análisis conjunto</i>	20
<i>Interpretación</i>	21
CONCLUSIONES	22
REFERENCIAS	23

Introducción

El Procesamiento del Lenguaje Natural (PLN) es una de las áreas más representativas de la Inteligencia Artificial, ya que busca desarrollar técnicas que permitan a las computadoras comprender, interpretar y generar lenguaje humano de manera automática. Gracias a los avances en aprendizaje automático y aprendizaje profundo, el PLN ha encontrado aplicaciones en asistentes virtuales, motores de búsqueda, traductores automáticos, sistemas de recomendación, análisis de opiniones y herramientas de apoyo para la toma de decisiones en distintos sectores.

Una de las tareas más comunes dentro del PLN es la **clasificación de sentimientos**, cuyo objetivo consiste en identificar automáticamente si un texto expresa una opinión positiva, negativa o neutra. Este tipo de análisis tiene un amplio uso en áreas como el comercio electrónico, las redes sociales, el servicio al cliente y el análisis de reputación digital, permitiendo transformar grandes volúmenes de información no estructurada en conocimiento útil para la toma de decisiones. Debido a ello, la clasificación de sentimientos se ha convertido en una de las tareas fundamentales del PLN y continúa siendo ampliamente utilizada como problema de referencia para evaluar nuevos algoritmos y técnicas de aprendizaje automático.

En este proyecto se utilizó el conjunto de datos **movie_reviews** de la biblioteca NLTK, el cual contiene reseñas de películas previamente clasificadas como positivas o negativas. A partir de este conjunto de datos se desarrollaron dos enfoques de clasificación: un modelo tradicional basado en **Naive Bayes** y un modelo de **red neuronal sencilla implementada con Keras**. La finalidad no es únicamente obtener el mejor desempeño posible, sino analizar las fortalezas y limitaciones de ambos enfoques cuando se enfrentan al mismo problema de clasificación de texto.

Para ello, se realizó un proceso de preprocesamiento del lenguaje que incluyó la normalización del texto, eliminación de palabras vacías y tokenización, seguido del entrenamiento y evaluación de ambos modelos mediante métricas como **accuracy**, **precision**, **recall**, **F1-score** y **ROC-AUC**. Asimismo, se incorporaron visualizaciones que facilitarán la interpretación de los resultados obtenidos.

Finalmente, este proyecto representa el cierre del módulo de Procesamiento del Lenguaje Natural del Postgrado en Inteligencia Artificial y Machine Learning de IEBS Digital School. Además de comparar dos enfoques ampliamente utilizados en tareas de clasificación de texto, el trabajo permite integrar los conocimientos adquiridos durante el curso y reflexionar sobre la importancia del PLN como una disciplina fundamental dentro de la Inteligencia Artificial y su potencial de aplicación en escenarios reales.

Aunque el conjunto de datos utilizado corresponde a reseñas de películas, las técnicas desarrolladas durante este proyecto pueden extrapolarse a numerosos dominios donde exista información textual. Las herramientas empleadas representan la base de múltiples aplicaciones actuales del Procesamiento del Lenguaje Natural y constituyen un punto de partida para el desarrollo de soluciones más especializadas.

Objetivo general

Desarrollar y evaluar un sistema de clasificación de sentimientos aplicado a reseñas de películas mediante técnicas de Procesamiento del Lenguaje Natural (PLN), comparando el desempeño de un modelo probabilístico basado en Naive Bayes y una red neuronal sencilla implementada con Keras, con el propósito de analizar sus fortalezas, limitaciones y comportamiento utilizando un conjunto de métricas de evaluación y técnicas de visualización aplicadas de manera uniforme a ambos modelos.

Objetivos específicos

- Preparar el conjunto de datos **movie_reviews** mediante un proceso de preprocesamiento que facilite su utilización en tareas de clasificación de texto.
- Implementar un modelo de clasificación basado en **Naive Bayes** como línea base del proyecto.
- Diseñar y entrenar una **red neuronal sencilla con Keras**, utilizando el mismo conjunto de datos para realizar una comparación objetiva entre ambos enfoques.
- Evaluar y comparar el desempeño de los modelos mediante métricas como *accuracy*, *precision*, *recall*, *F1-score* y *ROC-AUC*, identificando las fortalezas y limitaciones de cada uno.
- Analizar e interpretar los resultados obtenidos mediante visualizaciones que faciliten la comprensión del comportamiento de ambos modelos.
- Reflexionar sobre los conocimientos adquiridos durante el desarrollo del proyecto y sobre el potencial del Procesamiento del Lenguaje Natural en la resolución de problemas reales relacionados con el análisis de información textual.

Metodología

El desarrollo de este proyecto siguió una metodología experimental orientada a comparar el desempeño de dos enfoques de clasificación de sentimientos aplicados a un mismo conjunto de datos. Con el propósito de garantizar una comparación objetiva, ambos modelos fueron entrenados y evaluados bajo las mismas condiciones de preprocesamiento, partición de datos y métricas de evaluación.

En primer lugar, se utilizó el corpus **movie_reviews** disponible en la biblioteca NLTK, el cual contiene reseñas de películas previamente clasificadas como positivas o negativas. Este conjunto de datos fue seleccionado por tratarse de uno de los corpus de referencia más utilizados en tareas de clasificación de texto y análisis de sentimientos, permitiendo comparar resultados con otros trabajos desarrollados en el ámbito del Procesamiento del Lenguaje Natural.

Posteriormente, se realizó un proceso de preprocesamiento de los textos con el objetivo de reducir el ruido presente en la información y obtener una representación adecuada para el entrenamiento de los modelos. Entre las actividades desarrolladas se incluyen la normalización del texto, la tokenización, la eliminación de palabras vacías (*stopwords*) y la vectorización mediante técnicas de representación vectorial compatibles con algoritmos de aprendizaje automático.

Una vez preparados los datos, se implementaron dos modelos de clasificación. El primero corresponde a un clasificador probabilístico basado en **Naive Bayes**, ampliamente utilizado como modelo de referencia en tareas de clasificación de texto debido a su simplicidad y eficiencia computacional. El segundo consiste en una **red neuronal sencilla desarrollada con Keras**, cuyo propósito es analizar el comportamiento de un enfoque basado en aprendizaje profundo frente a un modelo clásico utilizando exactamente el mismo conjunto de entrenamiento y prueba.

Para evaluar ambos modelos se emplearon métricas ampliamente aceptadas en problemas de clasificación binaria, incluyendo *accuracy*, *precision*, *recall*, *F1-score* y *ROC-AUC*. Estas métricas permiten analizar no solamente la precisión global de cada modelo, sino también su capacidad para identificar correctamente cada una de las clases y comparar su comportamiento desde diferentes perspectivas.

Finalmente, los resultados obtenidos fueron complementados mediante visualizaciones gráficas que facilitaron su interpretación y permitieron identificar las principales fortalezas y limitaciones de cada enfoque. Esta metodología proporciona un proceso reproducible que permite evaluar objetivamente el impacto de diferentes técnicas de clasificación dentro del contexto del Procesamiento del Lenguaje Natural.

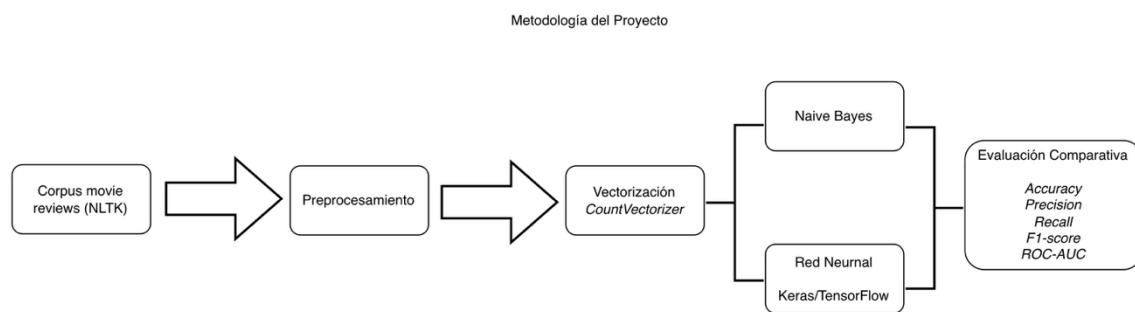


Figura 1. Flujo metodológico del proyecto

Descripción del Dataset

Descripción General

Para el desarrollo de este proyecto se utilizó el corpus **movie_reviews**, disponible como parte de la biblioteca **Natural Language Toolkit (NLTK)**. Este conjunto de datos constituye uno de los corpus de referencia más utilizados en tareas de clasificación de texto y análisis de sentimientos dentro del ámbito del Procesamiento del Lenguaje Natural.

El corpus está formado por reseñas de películas escritas en idioma inglés, previamente clasificadas en dos categorías: **positivas** y **negativas**. Cada documento contiene texto libre, lo que permite aplicar diferentes técnicas de preprocesamiento, representación vectorial y clasificación automática de sentimientos.

Una de las principales ventajas de este conjunto de datos es que presenta una distribución equilibrada entre ambas clases, lo que facilita la evaluación objetiva del desempeño de distintos algoritmos de clasificación sin introducir sesgos derivados del desbalance de datos. Además, al formar parte de NLTK, constituye un recurso ampliamente documentado y utilizado con fines académicos, favoreciendo la reproducibilidad de los experimentos.

Característica	Descripción
Nombre del corpus	<i>movie reviews</i>
Biblioteca	NLTK (Natural Language Toolkit)
Tipo de problema	Clasificación binaria de sentimientos
Idioma	Inglés
Número total de documentos	2,000
Clase positiva	1,000

Clase negativa	1,000
Formato	Texto libre
Dominio	Reseñas de películas
Etiquetas	Positive / Negative

Relevancia del corpus

El corpus *movie_reviews* constituye uno de los conjuntos de datos más utilizados en proyectos académicos de Procesamiento del Lenguaje Natural. Su equilibrio entre clases, disponibilidad dentro de NLTK y amplia documentación permiten realizar experimentos reproducibles y comparar los resultados obtenidos con investigaciones previas. Asimismo, su estructura resulta adecuada para implementar y evaluar tanto modelos clásicos de aprendizaje automático como enfoques basados en redes neuronales. Estas características hacen que el corpus resulte adecuado para el desarrollo de las actividades descritas en la Fase 1 del presente proyecto.

Limitaciones del dataset

Aunque el corpus *movie_reviews* resulta adecuado para fines académicos, presenta algunas limitaciones. En primer lugar, las reseñas pertenecen exclusivamente al dominio cinematográfico, por lo que el vocabulario y el contexto pueden diferir significativamente de otros ámbitos de aplicación. Asimismo, el corpus únicamente contempla dos categorías de sentimiento (positivo y negativo), sin considerar opiniones neutras o emociones más complejas. Finalmente, al tratarse de un conjunto de datos relativamente pequeño en comparación con los grandes corpus utilizados actualmente para entrenar modelos de lenguaje, los resultados obtenidos deben interpretarse dentro del contexto experimental del proyecto.

Desarrollo

Fase 1: Preparación de los datos

Objetivo

El objetivo de esta primera fase consistió en preparar el conjunto de datos *movie_reviews* para su utilización en los modelos de clasificación desarrollados posteriormente. Para ello, fue necesario cargar el corpus desde la biblioteca NLTK, explorar su estructura, identificar la distribución de las clases y aplicar un conjunto de técnicas de preprocesamiento orientadas a reducir el ruido presente en los textos.

El preprocesamiento constituye una de las etapas más importantes dentro del Procesamiento del Lenguaje Natural, ya que la calidad de los datos de entrada influye directamente en el desempeño de los modelos de aprendizaje automático. En esta fase se buscó transformar las reseñas originales en una representación más adecuada para su posterior vectorización y entrenamiento.

Actividades

Durante esta fase se llevaron a cabo las siguientes actividades:

1. Descarga y carga del corpus **movie_reviews** desde la biblioteca NLTK.
2. Exploración inicial del conjunto de datos para verificar la cantidad de documentos y la distribución de las clases.
3. Conversión de las reseñas a una estructura de datos adecuada para su procesamiento.
4. Aplicación de técnicas de preprocesamiento, incluyendo:
 - Conversión del texto a minúsculas.
 - Tokenización.
 - Eliminación de palabras vacías (*stopwords*).
 - Limpieza de caracteres innecesarios.
5. Preparación del conjunto de datos para su representación vectorial mediante **CountVectorizer**.

Implementación

En esta fase se emplearon las herramientas proporcionadas por la biblioteca **NLTK** para acceder al corpus **movie_reviews**, así como las funciones de preprocesamiento necesarias para transformar el texto libre en una representación adecuada para los algoritmos de clasificación.

Posteriormente, los documentos fueron preparados para su vectorización utilizando **CountVectorizer** de Scikit-learn, el cual convierte cada documento en una representación numérica basada en la frecuencia de aparición de las palabras. Esta representación constituye la entrada utilizada por los modelos desarrollados en las fases posteriores.

Resultados

Como resultado de esta fase se obtuvo un conjunto de datos correctamente estructurado y preprocesado, listo para ser utilizado durante el entrenamiento de los modelos de clasificación.

Asimismo, la exploración inicial permitió confirmar que el corpus presenta una distribución equilibrada entre las clases positiva y negativa, condición que favorece una comparación objetiva entre los distintos algoritmos implementados.

Las Figuras 2 a 6 muestran de manera secuencial las principales actividades realizadas durante la preparación del conjunto de datos. En primer lugar, se confirma la correcta descarga del corpus *movie_reviews* y su incorporación al entorno de trabajo. Posteriormente, la exploración inicial permitió verificar que el conjunto de datos está compuesto por **2,000 reseñas**, distribuidas de forma equilibrada entre las clases **positiva** y **negativa**, condición que favorece una evaluación objetiva de los modelos.

Asimismo, el proceso de preprocesamiento permitió transformar las reseñas originales mediante la normalización del texto, la eliminación de palabras vacías y la limpieza de caracteres innecesarios, obteniendo una representación más consistente para su análisis. Finalmente, la vectorización mediante **CountVectorizer** generó una matriz de entrenamiento de **1600 × 30000** y una matriz de prueba de **400 × 30000**, utilizando un vocabulario máximo de **30000 términos**. Esta transformación convirtió el texto libre en una representación numérica adecuada para el entrenamiento de los modelos desarrollados en las fases posteriores.

```
[3]: recursos_nltk = [
    "movie_reviews",
    "stopwords",
    "punkt",
    "punkt_tab"
]

for recurso in recursos_nltk:
    nltk.download(recurso, quiet=True)

print("Recursos de NLTK descargados correctamente.")
Recursos de NLTK descargados correctamente.
```

Figura 2. Descarga del corpus

```
[4]: reviews = [
    (movie_reviews.raw(fileid), category)
    for category in movie_reviews.categories()
    for fileid in movie_reviews.fileids(category)
]

df = pd.DataFrame(reviews, columns=["review", "category"])
df["sentiment"] = df["category"].map({
    "pos": "positivo",
    "neg": "negativo"
})

print("Dimensiones del dataset:", df.shape)
display(df.head())
```

Dimensiones del dataset: (2000, 3)

	review	category	sentiment
0	plot : two teen couples go to a church party ,...	neg	negativo
1	the happy bastard's quick movie review \ndamn ...	neg	negativo
2	it is movies like these that make a jaded movi...	neg	negativo
3	" quest for camelot " is warner bros . ' firs...	neg	negativo
4	synopsis : a mentally unstable man undergoing ...	neg	negativo

Figura 3. Número de documentos

```
[5]: print("Distribución de clases:")
display(df["sentiment"].value_counts().rename_axis("sentimiento").to_frame("cantidad"))

print("\nValores nulos:")
display(df.isna().sum().to_frame("cantidad"))

print("\nEjemplo de reseña:")
print(df.loc[0, "review"][:700])
```

Distribución de clases:

	cantidad
sentimiento	
negativo	1000
positivo	1000

Valores nulos:

	cantidad
review	0
category	0
sentiment	0

Ejemplo de reseña:

plot : two teen couples go to a church party , drink and then drive .
they get into an accident .
one of the guys dies , but his girlfriend continues to see him in her life , and has nightmares .
what's the deal ?
watch the movie and " sorta " find out . . .
critique : a mind-fuck movie for the teen generation that touches on a very cool idea , but presents it in a very bad package .
which is what makes this review an even harder one to write , since i generally applaud films which attempt to break the mold , mess with yo
ur head and such (lost highway & memento) , but there are good and bad ways of making all types of films , and these folks just didn't sna
g this one correctly .
the

Figura 4. Distribución de clases y primeras reseñas

```
[6]: stop_words = set(stopwords.words("english"))

def preprocess_text(text):
    """Limpia y tokeniza una reseña en inglés."""
    text = text.lower()
    text = re.sub(r"\d+", " ", text)
    text = text.translate(str.maketrans("", "", string.punctuation))

    tokens = word_tokenize(text)
    tokens = [
        token
        for token in tokens
        if token.isalpha()
        and token not in stop_words
        and len(token) > 1
    ]

    return " ".join(tokens)

df["clean_review"] = df["review"].apply(preprocess_text)

display(df[["sentiment", "review", "clean_review"]].head(3))
```

	sentiment	review	clean_review
0	negativo	plot : two teen couples go to a church party ,...	plot two teen couples go church party drink dr...
1	negativo	the happy bastard's quick movie review \ndamn ...	happy bastards quick movie review damn bug got...
2	negativo	it is movies like these that make a jaded movi...	movies like make jaded movie viewer thankful i...

Figura 5. Resultado del preprocesamiento

```
[7]: print("Texto original:")
print(df.loc[0, "review"][:500])

print("\nTexto preprocesado:")
print(df.loc[0, "clean_review"][:500])

Texto original:
plot : two teen couples go to a church party , drink and then drive .
they get into an accident .
one of the guys dies , but his girlfriend continues to see him in her life , and has nightmares .
what's the deal ?
watch the movie and " sorta " find out . . .
critique : a mind-fuck movie for the teen generation that touches on a very cool idea , but presents it in a very bad package .
which is what makes this review an even harder one to write , since i generally applaud films which attempt

Texto preprocesado:
plot two teen couples go church party drink drive get accident one guys dies girlfriend continues see life nightmares whats deal watch movie
sorta find critique mindfuck movie teen generation touches cool idea presents bad package makes review even harder one write since generally
applaud films attempt break mold mess head lost highway memento good bad ways making types films folks didnt snag one correctly seem taken p
retty neat concept executed terribly problems movie well main problem simply j
```

```
[9]: vectorizer = CountVectorizer(
    min_df=2,
    max_df=0.95,
    ngram_range=(1, 2),
    max_features=30000
)

X_train_vectorized = vectorizer.fit_transform(X_train)
X_test_vectorized = vectorizer.transform(X_test)

print("Forma de la matriz de entrenamiento:", X_train_vectorized.shape)
print("Forma de la matriz de prueba:", X_test_vectorized.shape)
print("Tamaño del vocabulario:", len(vectorizer.get_feature_names_out()))

Forma de la matriz de entrenamiento: (1600, 30000)
Forma de la matriz de prueba: (400, 30000)
Tamaño del vocabulario: 30000
```

Figura 6. Representación vectorial del conjunto de datos utilizando *CountVectorizer*

Interpretación

Los resultados obtenidos durante esta fase evidencian la importancia del preprocesamiento dentro de los proyectos de Procesamiento del Lenguaje Natural. Aunque las reseñas originales contienen información útil para la clasificación de

sentimientos, también presentan elementos que pueden introducir ruido durante el entrenamiento de los modelos.

La aplicación de técnicas como la tokenización, la eliminación de *stopwords*, la normalización del texto y la vectorización mediante **CountVectorizer** permitió obtener una representación consistente de los documentos, transformando el texto libre en una estructura numérica adecuada para los algoritmos de clasificación. Asimismo, la exploración inicial confirmó que el corpus presenta una distribución balanceada entre las clases positiva y negativa, condición que favorece una comparación objetiva entre ambos modelos y establece las bases para las fases de entrenamiento y evaluación comparativa del proyecto.

Fase 2: Entrenamiento de Modelos

Objetivo

El objetivo de esta fase consistió en entrenar dos modelos de clasificación de sentimientos utilizando el mismo conjunto de datos previamente preparado durante la Fase 1. Con ello se buscó comparar el comportamiento de un modelo clásico de aprendizaje automático, representado por Naive Bayes, frente a una red neuronal sencilla implementada con Keras. Ambos modelos fueron entrenados utilizando exactamente el mismo conjunto de entrenamiento y la misma representación vectorial de los textos, garantizando así una comparación objetiva de su desempeño en la fase de evaluación.

Actividades

Durante esta fase se desarrollaron las siguientes actividades:

1. Entrenamiento del modelo probabilístico **Multinomial Naive Bayes** utilizando la representación vectorial generada mediante *CountVectorizer*.
2. Obtención de las probabilidades de clasificación para cada documento del conjunto de prueba.
3. Diseño de una red neuronal densa utilizando la biblioteca Keras.
4. Configuración de la arquitectura del modelo, incluyendo capas densas, función de activación ReLU, capa de salida sigmoide y optimizador Adam.
5. Entrenamiento de la red neuronal utilizando los mismos datos empleados por Naive Bayes.

6. Aplicación del mecanismo **EarlyStopping** para evitar el sobreentrenamiento y conservar automáticamente la mejor versión del modelo durante el proceso de aprendizaje.

Implementación

En esta fase se entrenaron dos modelos diferentes con el propósito de comparar un enfoque clásico de clasificación basado en probabilidades frente a un enfoque basado en aprendizaje profundo. Para asegurar la validez de la comparación, ambos modelos utilizaron exactamente la misma partición de entrenamiento y prueba, así como la misma representación vectorial obtenida mediante *CountVectorizer* durante la fase anterior.

El primer modelo corresponde a un clasificador **Multinomial Naive Bayes**, ampliamente utilizado como línea base en problemas de clasificación de texto debido a su sencillez, rapidez de entrenamiento y buen desempeño en tareas de Procesamiento del Lenguaje Natural.

Posteriormente, se implementó una **red neuronal densa con Keras**, cuya arquitectura fue diseñada para recibir como entrada los vectores generados por *CountVectorizer*. El modelo incorpora funciones de activación ReLU en las capas ocultas y una función sigmoide en la capa de salida para realizar la clasificación binaria entre reseñas positivas y negativas. Asimismo, se utilizó el optimizador Adam y la función de pérdida Binary Crossentropy, configuraciones ampliamente utilizadas en problemas de clasificación supervisada.

2.1. Naive Bayes

El entrenamiento del modelo Naive Bayes se realizó utilizando la implementación **MultinomialNB** de Scikit-learn. Este algoritmo estima la probabilidad de pertenencia de cada documento a una determinada clase a partir de la frecuencia de aparición de las palabras presentes en la representación vectorial del corpus. Debido a su eficiencia computacional, Naive Bayes constituye una de las alternativas más utilizadas como modelo base en tareas de clasificación de texto.

```
[10]: model = MultinomialNB()
      model.fit(X_train_vectorized, y_train)

      y_pred = model.predict(X_test_vectorized)

      print("Modelo entrenado correctamente.")
      Modelo entrenado correctamente.
```

Figura 7. Entrenamiento del modelo Multinomial Naive Bayes

La Figura 7 muestra que el modelo Multinomial Naive Bayes fue entrenado correctamente utilizando el conjunto de entrenamiento previamente vectorizado. Una vez finalizado este proceso, el modelo quedó preparado para generar predicciones y estimar las probabilidades asociadas a cada documento del conjunto de prueba, información que será utilizada posteriormente durante la evaluación comparativa.

2.2. Red Neuronal

Con el propósito de comparar el desempeño de un modelo basado en aprendizaje profundo, se implementó una red neuronal sencilla utilizando Keras. La arquitectura seleccionada está compuesta por una capa de entrada compatible con la representación vectorial del texto, dos capas densas con función de activación ReLU y una capa de salida con función sigmoide para realizar la clasificación binaria.

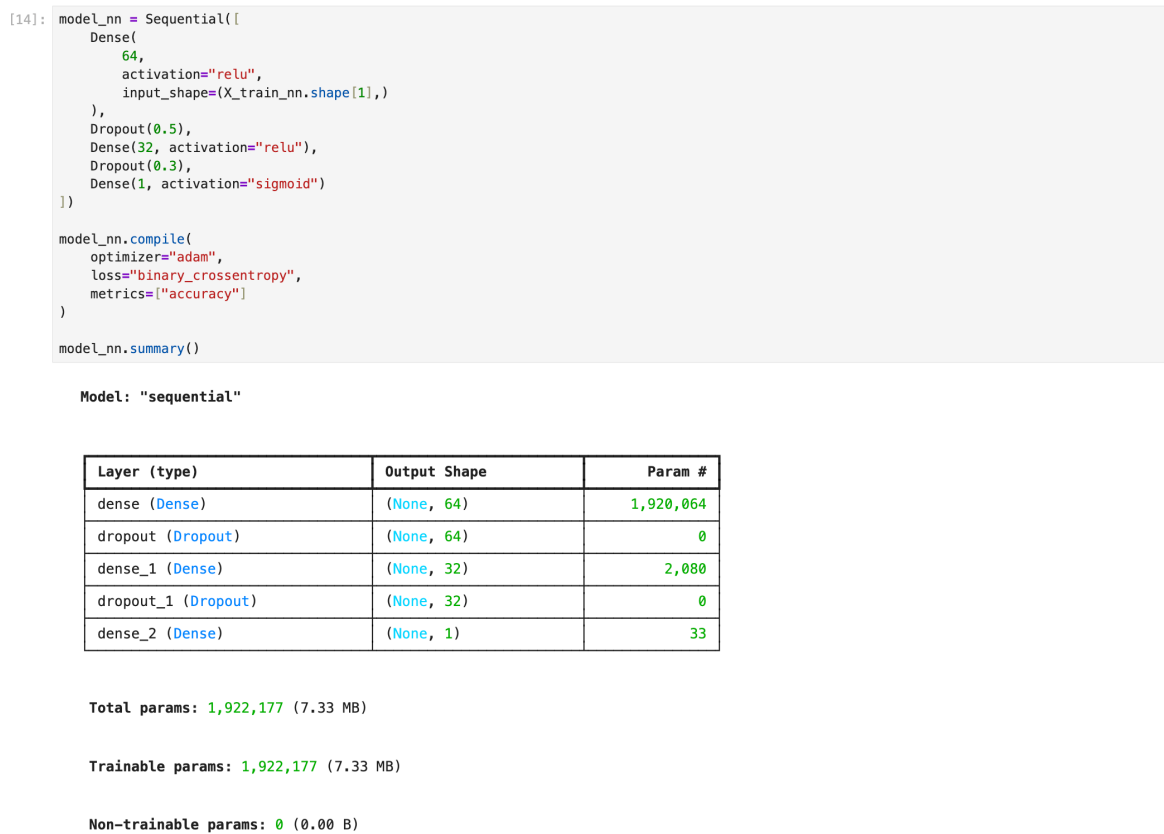


Figura 8. Arquitectura del modelo de red neuronal implementado con Keras

La arquitectura presentada en la Figura 8 muestra una red neuronal de baja complejidad, adecuada para el tamaño del conjunto de datos utilizado. Está compuesta por dos capas densas con funciones de activación ReLU y una capa de

salida con activación sigmoide para realizar la clasificación binaria. Esta configuración ofrece un equilibrio entre capacidad de aprendizaje y costo computacional, permitiendo identificar patrones relevantes en las reseñas sin recurrir a arquitecturas profundas más complejas.

Posteriormente se inició el proceso de entrenamiento utilizando el conjunto de entrenamiento previamente preparado. Durante este proceso se aplicó la estrategia **EarlyStopping**, la cual supervisa el desempeño del modelo sobre los datos de validación y detiene automáticamente el entrenamiento cuando deja de observarse una mejora significativa.

```
[15]: early_stopping = EarlyStopping(
      monitor="val_loss",
      patience=3,
      restore_best_weights=True
    )

    historial_nn = model_nn.fit(
        X_train_nn,
        y_train_nn,
        validation_split=0.20,
        epochs=15,
        batch_size=32,
        callbacks=[early_stopping],
        verbose=1
    )
```

Epoch 1/15	40/40	1s	8ms/step	- accuracy: 0.6656	- loss: 0.6085	- val_accuracy: 0.8406	- val_loss: 0.4630
Epoch 2/15	40/40	0s	5ms/step	- accuracy: 0.9047	- loss: 0.2762	- val_accuracy: 0.8656	- val_loss: 0.4020
Epoch 3/15	40/40	0s	4ms/step	- accuracy: 0.9766	- loss: 0.0988	- val_accuracy: 0.8594	- val_loss: 0.4457
Epoch 4/15	40/40	0s	6ms/step	- accuracy: 0.9898	- loss: 0.0466	- val_accuracy: 0.8375	- val_loss: 0.5475
Epoch 5/15	40/40	0s	5ms/step	- accuracy: 0.9953	- loss: 0.0307	- val_accuracy: 0.8438	- val_loss: 0.5699

Figura 9. Historial de entrenamiento utilizando *EarlyStopping*

Como se aprecia en la Figura 9, el historial de entrenamiento muestra la evolución del modelo a lo largo de las diferentes épocas. La estrategia *EarlyStopping* supervisó el desempeño sobre el conjunto de validación y detuvo automáticamente el entrenamiento cuando dejó de observarse una mejora significativa, reduciendo el riesgo de sobreajuste y conservando los pesos correspondientes a la mejor época alcanzada.

Resultados

Como resultado de esta fase se obtuvieron dos modelos completamente entrenados y preparados para ser evaluados utilizando el mismo conjunto de prueba. Además de las predicciones finales, ambos modelos generaron las probabilidades asociadas a cada clasificación, información indispensable para calcular métricas de desempeño como *Accuracy*, *Precision*, *Recall*, *F1-score* y *ROC-AUC* durante la siguiente fase del proyecto.

Interpretación

El entrenamiento de ambos modelos bajo las mismas condiciones experimentales permitió establecer una base sólida para realizar una comparación objetiva durante la fase de evaluación. Mientras que Naive Bayes ofreció una implementación sencilla, rápida y con un costo computacional reducido, la red neuronal incorporó una arquitectura más flexible capaz de aprender relaciones no lineales presentes en el texto mediante un proceso iterativo de optimización.

Aunque ambos modelos persiguen el mismo objetivo de clasificación, sus principios de funcionamiento son diferentes. Naive Bayes basa sus predicciones en relaciones probabilísticas entre las palabras del documento, mientras que la red neuronal ajusta automáticamente sus parámetros durante el entrenamiento para identificar patrones más complejos. Esta diferencia metodológica constituye el punto de partida para analizar sus fortalezas y limitaciones en la siguiente fase del proyecto, donde ambos enfoques serán comparados utilizando métricas cuantitativas y visualizaciones específicas.

Fase 3: Evaluación comparativa

Objetivo

El objetivo de esta fase consistió en evaluar y comparar objetivamente el desempeño de los modelos entrenados durante la fase anterior utilizando un mismo conjunto de prueba. Para ello se calcularon diferentes métricas de clasificación, se analizaron las matrices de confusión y se construyó una curva ROC comparativa que permitió identificar las fortalezas y limitaciones de cada enfoque en la tarea de clasificación de sentimientos.

Actividades

Durante esta fase se realizaron las siguientes actividades:

1. Generación de las predicciones de ambos modelos sobre el conjunto de prueba.
2. Obtención de las probabilidades de clasificación.
3. Cálculo de *Accuracy*.
4. Cálculo de *Precision*.

5. Cálculo de *Recall*.
6. Cálculo de *F1-score*.
7. Cálculo del área bajo la curva ROC (*ROC-AUC*).
8. Construcción de matrices de confusión.
9. Comparación de resultados mediante tablas y gráficas.

Implementación

Una vez entrenados ambos modelos, se utilizaron las reseñas del conjunto de prueba para generar las predicciones correspondientes. A partir de estas predicciones y de las probabilidades estimadas por cada modelo se calcularon diversas métricas de evaluación ampliamente utilizadas en problemas de clasificación binaria. Asimismo, se construyeron matrices de confusión para analizar el comportamiento de cada clasificador sobre ambas categorías y se generó una curva ROC comparativa para evaluar su capacidad de discriminación.

Resultados

	Modelo	Accuracy	Precision	Recall	F1-score	ROC-AUC
0	Naive Bayes	0.8100	0.7925	0.840	0.8155	0.8870
1	Red Neuronal	0.8275	0.7835	0.905	0.8399	0.9175

Figura 10. Tabla comparativa

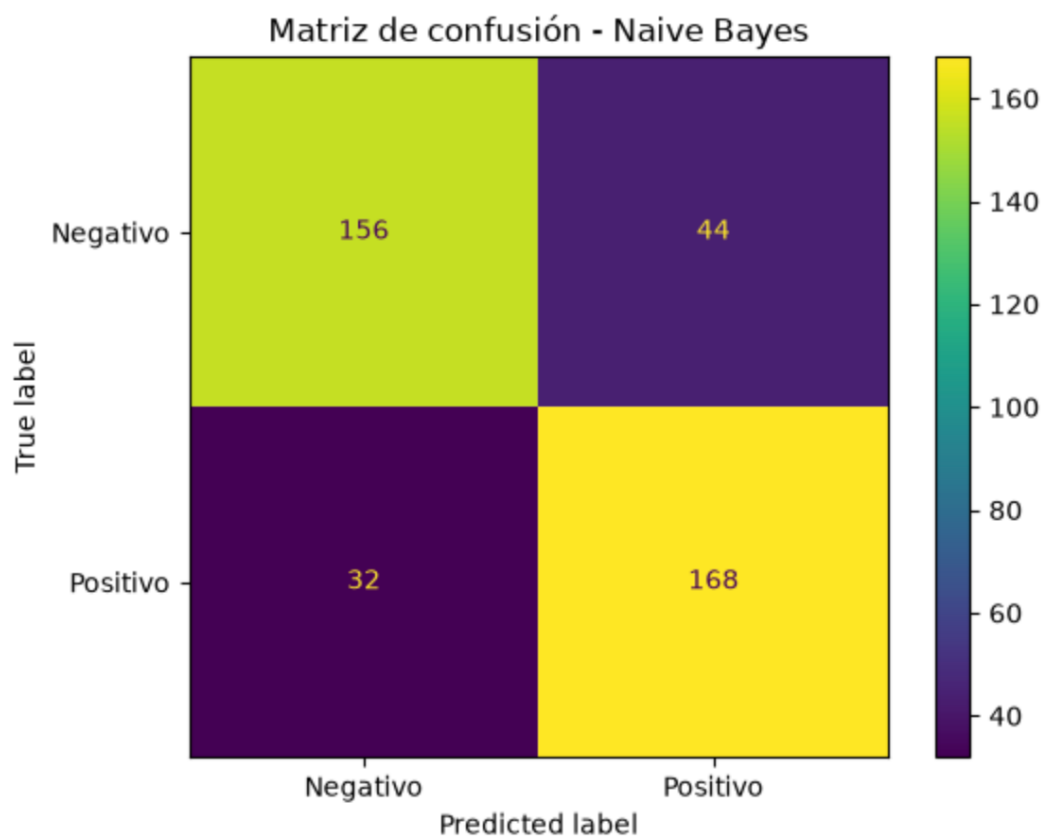


Figura 11. Matriz de confusión del modelo Naive Bayes

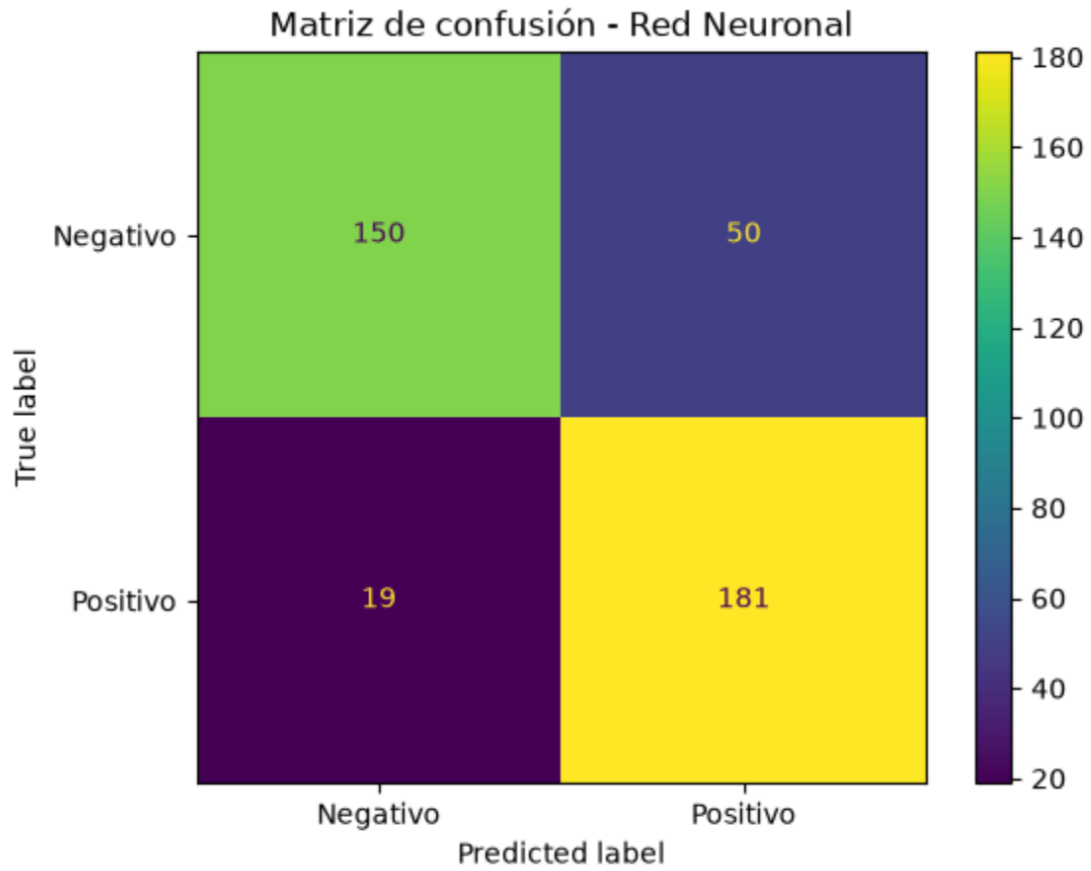


Figura 12. Matriz de confusión de la Red Neuronal

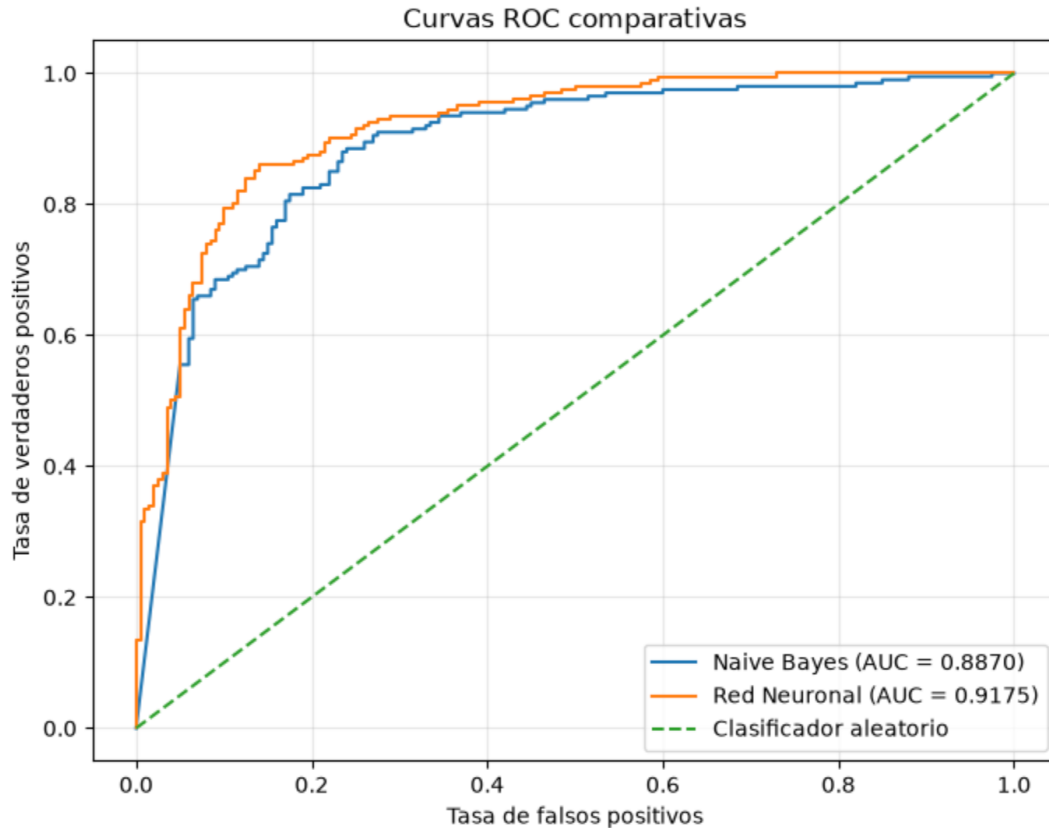


Figura 13. Curvas ROC comparativas

Análisis conjunto

Los resultados obtenidos muestran que ambos modelos fueron capaces de clasificar correctamente una proporción elevada de las reseñas del conjunto de prueba. Sin embargo, el análisis conjunto de las métricas evidencia diferencias relevantes en su comportamiento.

La **Figura 10** permite observar las diferencias entre ambos enfoques considerando *Accuracy*, *Precision*, *Recall*, *F1-score* y *ROC-AUC*. Estas métricas proporcionan una visión integral del desempeño de los modelos y permiten evaluar tanto la precisión global como la capacidad para identificar correctamente cada una de las clases.

Las **matrices de confusión (Figuras 11 y 12)** complementan este análisis al mostrar la distribución de aciertos y errores para cada modelo. Gracias a esta representación es posible identificar no sólo el número de clasificaciones correctas, sino también los falsos positivos y falsos negativos generados durante el proceso de evaluación.

Finalmente, la **curva ROC comparativa (Figura 13)** resume la capacidad discriminativa de ambos clasificadores utilizando diferentes umbrales de decisión. Un área bajo la curva (**ROC-AUC**) más elevada indica una mayor capacidad del modelo para distinguir correctamente entre reseñas positivas y negativas.

Interpretación

Los resultados obtenidos evidencian que ambos modelos fueron capaces de clasificar correctamente la mayoría de las reseñas del conjunto de prueba, alcanzando niveles de desempeño superiores al 80 % de *Accuracy*. No obstante, la **red neuronal** obtuvo el mejor rendimiento global al alcanzar un **Accuracy de 0.8275**, un **Recall de 0.9050**, un **F1-score de 0.8399** y un **ROC-AUC de 0.9175**, superando ligeramente al modelo **Naive Bayes**, que registró un **Accuracy de 0.8100**, un **F1-score de 0.8155** y un **ROC-AUC de 0.8870**. Estos resultados indican que la arquitectura neuronal logró una mejor capacidad para discriminar entre reseñas positivas y negativas utilizando la misma representación vectorial del texto.

El análisis de las matrices de confusión confirma este comportamiento. Mientras que **Naive Bayes** clasificó correctamente **324 de las 400 reseñas** del conjunto de prueba (156 negativas y 168 positivas), la **red neuronal** incrementó el número de clasificaciones correctas hasta **331 documentos** (150 negativas y 181 positivas). En particular, la red neuronal redujo de manera importante el número de **falsos negativos**, pasando de **32 a 19**, lo que explica el incremento observado en la métrica de *Recall*. A cambio, presentó un ligero aumento en los falsos positivos (50 frente a 44), comportamiento que también se refleja en una *Precision* ligeramente inferior respecto a Naive Bayes.

La comparación de las curvas ROC complementa esta interpretación. El valor de **ROC-AUC de 0.9175** obtenido por la red neuronal indica una mayor capacidad para distinguir correctamente entre ambas clases a diferentes umbrales de decisión, mientras que el **ROC-AUC de 0.8870** alcanzado por Naive Bayes confirma que, aun siendo un modelo considerablemente más sencillo desde el punto de vista computacional, mantiene un desempeño competitivo para tareas de clasificación de texto.

En conjunto, los resultados demuestran que una arquitectura neuronal relativamente simple puede ofrecer una mejora medible sobre un modelo probabilístico clásico cuando ambos utilizan el mismo proceso de preprocesamiento y la misma representación vectorial mediante **CountVectorizer**. No obstante, la diferencia entre ambos enfoques no es tan amplia como podría esperarse, lo que pone de manifiesto que **Naive Bayes continúa siendo una alternativa sólida como modelo base**

gracias a su rapidez de entrenamiento, simplicidad de implementación y capacidad para obtener resultados competitivos. La elección entre ambos enfoques dependerá del equilibrio que se busque entre desempeño predictivo, complejidad computacional y requisitos específicos de la aplicación.

Conclusiones

El desarrollo de este proyecto permitió aplicar de manera integral los conceptos estudiados durante el módulo de Procesamiento del Lenguaje Natural, recorriendo todas las etapas necesarias para construir un sistema de clasificación de sentimientos: desde la preparación del conjunto de datos hasta la evaluación comparativa de distintos modelos de aprendizaje automático. Más allá de los resultados obtenidos, el proyecto permitió comprender la importancia que tiene cada fase del proceso sobre el desempeño final de los modelos.

Durante la etapa de preparación de los datos se comprobó que el preprocesamiento constituye uno de los elementos más relevantes dentro de cualquier proyecto de PLN. Actividades como la normalización del texto, la eliminación de palabras vacías, la tokenización y la representación vectorial mediante **CountVectorizer** permitieron transformar información textual no estructurada en una representación numérica adecuada para el entrenamiento de algoritmos de clasificación. Este proceso estableció una base consistente para que ambos modelos fueran comparados bajo las mismas condiciones experimentales.

La comparación entre **Multinomial Naive Bayes** y una **red neuronal implementada con Keras** mostró que ambos enfoques son capaces de obtener resultados competitivos cuando se aplican sobre el mismo conjunto de datos. Aunque la red neuronal alcanzó el mejor desempeño global en la mayoría de las métricas de evaluación, la diferencia respecto a Naive Bayes resultó relativamente moderada. Este comportamiento confirma que los modelos probabilísticos continúan siendo una alternativa válida para tareas de clasificación de texto, especialmente cuando se busca simplicidad, rapidez de entrenamiento y bajo costo computacional.

Asimismo, el análisis de las matrices de confusión y de las curvas ROC permitió comprender que la evaluación de un modelo no debe limitarse únicamente al valor de *Accuracy*. La utilización conjunta de métricas como **Precision**, **Recall**, **F1-score** y **ROC-AUC** proporciona una visión mucho más completa del comportamiento de los clasificadores y facilita la identificación de sus fortalezas y limitaciones en diferentes escenarios.

Finalmente, este proyecto permitió consolidar los conocimientos adquiridos durante el módulo mediante la integración de herramientas ampliamente utilizadas dentro del ecosistema de Ciencia de Datos y Procesamiento del Lenguaje Natural, como **NLTK**, **Scikit-learn** y **TensorFlow/Keras**. En conjunto, los resultados obtenidos demuestran que una adecuada preparación de los datos, acompañada de una metodología experimental consistente y una evaluación objetiva mediante múltiples métricas, constituye un elemento fundamental para desarrollar soluciones de clasificación de texto confiables y reproducibles.

Referencias

- Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python*. O'Reilly Media.
- Chollet, F. (2021). *Deep learning with Python* (2nd ed.). Manning Publications.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Jurafsky, D., & Martin, J. H. (2025). *Speech and language processing* (3rd ed., draft). Stanford University. <https://web.stanford.edu/~jurafsky/slp3/>
- Keras. (s. f.). *Keras documentation*. <https://keras.io/>
- NLTK Project. (s. f.). *Movie reviews corpus*. <https://www.nltk.org/book/>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- Powers, D. M. W. (2011). Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, 2(1), 37–63.
- TensorFlow. (s. f.). *TensorFlow documentation*. <https://www.tensorflow.org/>